

EE 601L: Neural Networks and Deep Learning

Lab-2

Pakalapati Naga Srinivasa Varma

1 Dataset

I have taken 500 data points from make_circles dataset on which I had performed my analysis.

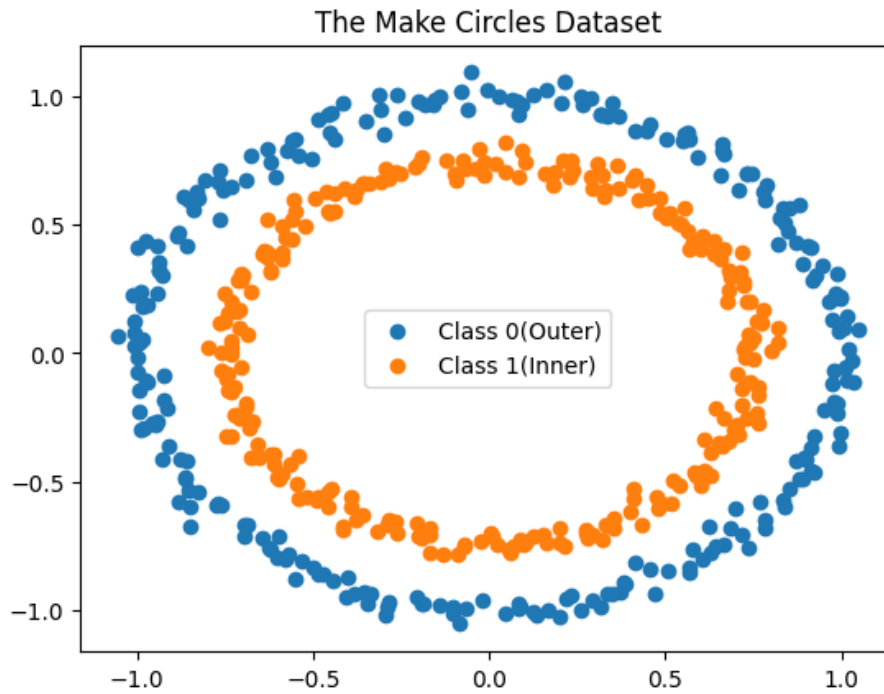
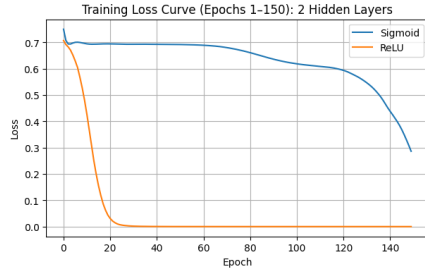


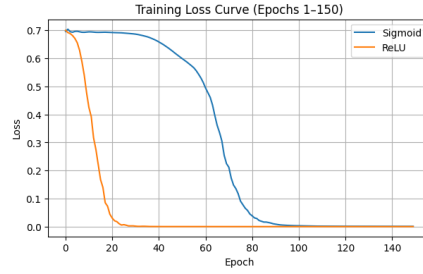
Figure 1: Plot of Data Points

2 Analysis and Observations

2.1 Comparison of Sigmoid and ReLU models:



(a) Training Loss Curve for 2 Hidden Layers



(b) Training Loss Curve for 3 Hidden Layers

Figure 2: Convergence behavior of the model

We can see that the loss curve of ReLU activation function reduces much faster than the sigmoid activation function.

This is because its gradient remains constant (equal to 1) for positive inputs, allowing efficient gradient flow during backpropagation. In contrast, sigmoid activations saturate for large positive or negative inputs, producing very small gradients. This vanishing gradient effect slows weight updates in sigmoid-based networks, requiring more epochs to reach stable convergence.

The model converges at around 20 epochs when ReLU activation is used, while sigmoid-based models converge around 100 epochs for the 3-hidden-layer architecture and take even longer for the 2-hidden-layer model.

2.2 Decision Boundaries and Accuracies:

The basic perceptron model can only realize linear decision boundaries; hence, the classification accuracy remains low.

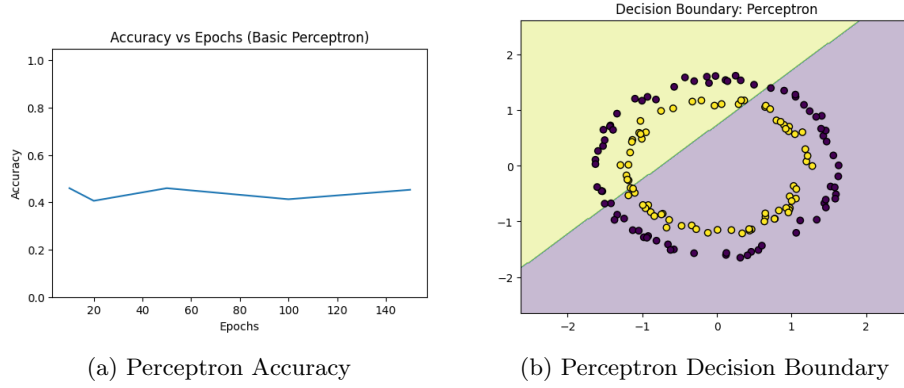


Figure 3: Basic Perceptron

To overcome this limitation, hidden layers were introduced by stacking multiple perceptrons. However, due to the use of a hard-limiting step activation function, the model still fails to learn non-linear decision boundaries effectively.

Multiple layers with step activation do not learn non-linear boundaries because the step function is non-differentiable, preventing backpropagation. Hidden-layer weights therefore do not adapt meaningfully during training. This causes the network to behave like a linear classifier despite increased depth.

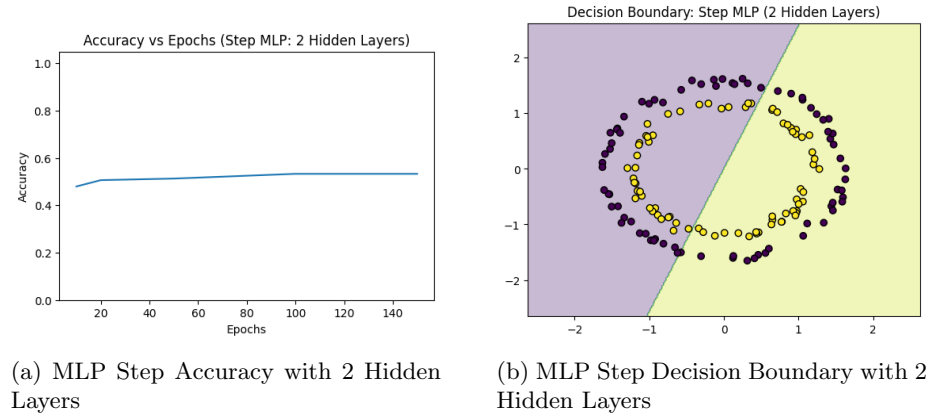


Figure 4: MLP with Step Activation Function and 2 Hidden Layers

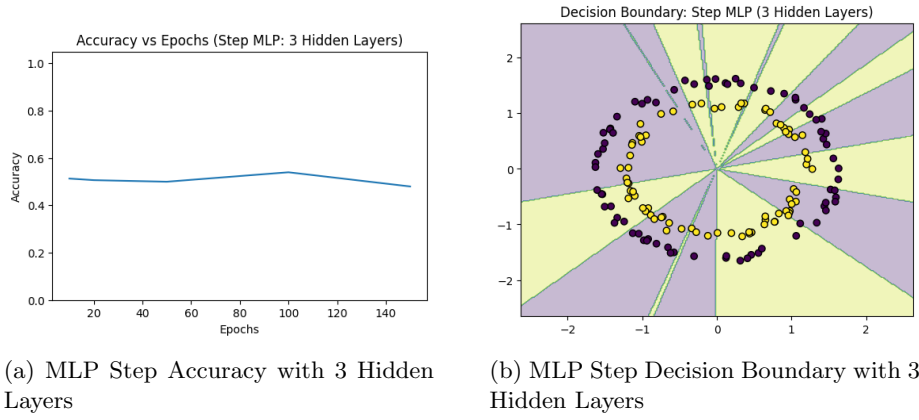


Figure 5: MLP with Step Activation Function and 3 Hidden Layers

Hence, we used non-linear activation functions to realize non-linear boundaries like Sigmoid and ReLU. However, Sigmoid activation function had the issue of vanishing gradient due to which, it took more epochs for convergence.

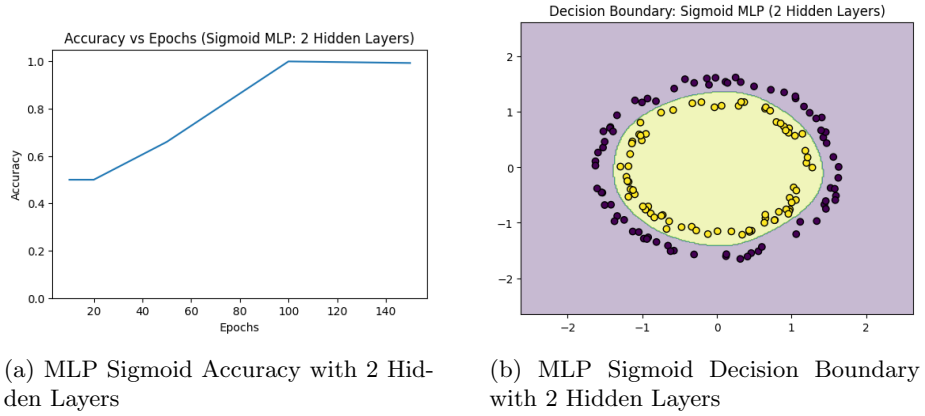


Figure 6: MLP with Sigmoid Activation Function and 2 Hidden Layers

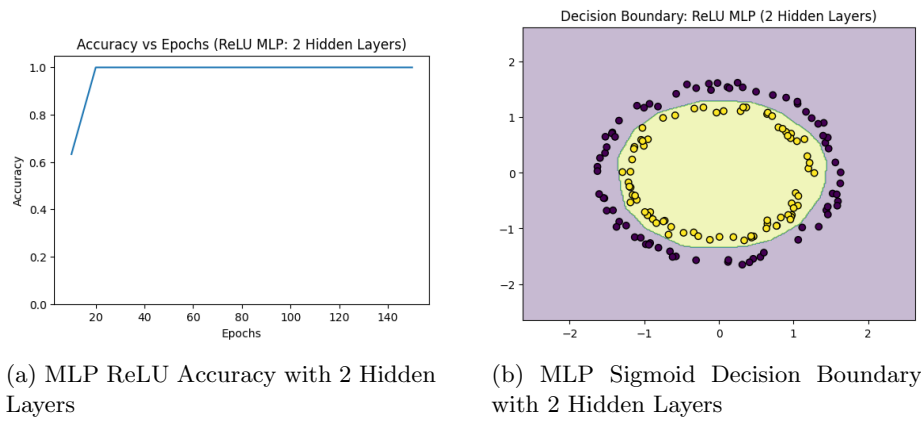


Figure 7: MLP with ReLU Activation Function and 2 Hidden Layers

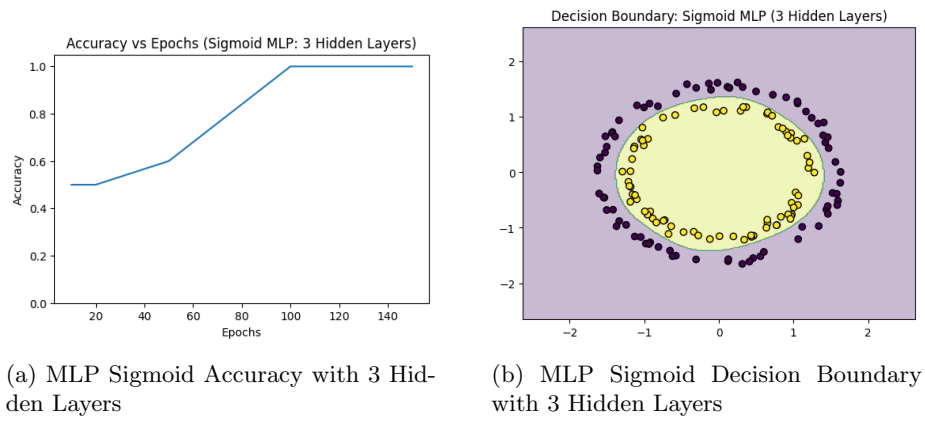


Figure 8: MLP with Sigmoid Activation Function and 3 Hidden Layers

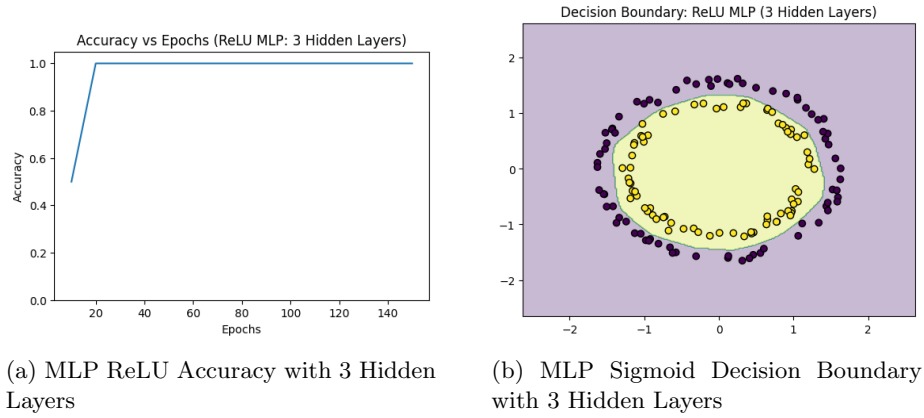


Figure 9: MLP with ReLU Activation Function and 3 Hidden Layers

The Gradients keep decreasing for sigmoid because of its formula of $\sigma(1-\sigma)$ which decrease the gradient a lot and results in slow convergence and longer time during back propagation.

The Gradients of ReLU are either 1 or 0 so the convergence is much faster. We can observe this in the loss vs epochs curve at the beginning.

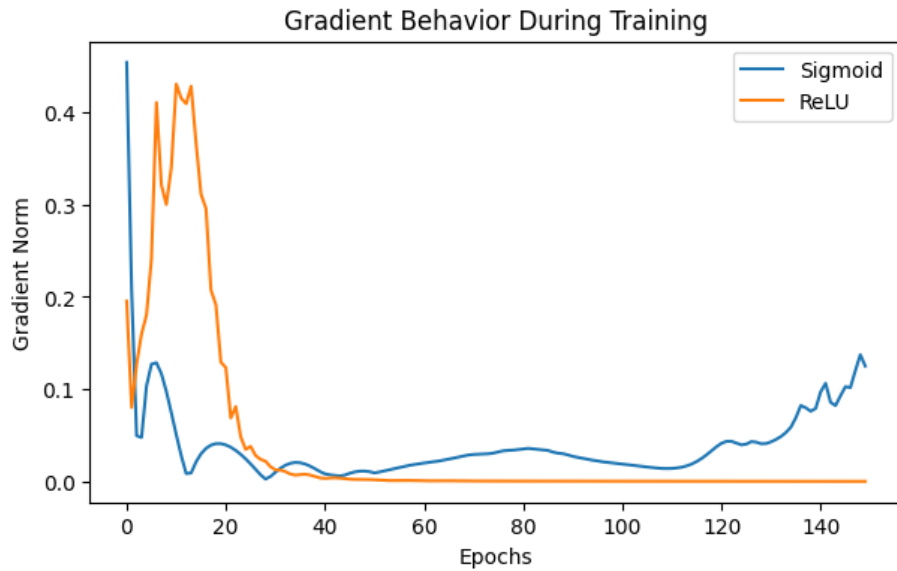


Figure 10: Gradient Behavior

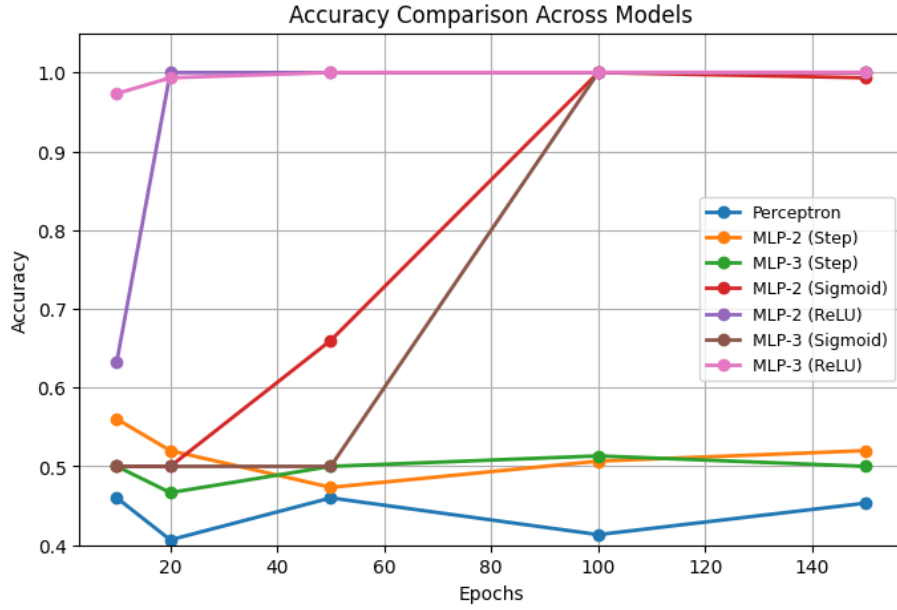


Figure 11: Accuracies Plot

Increasing network depth improves the ability to learn non-linear decision boundaries. The perceptron and step-based MLPs fail to model circular patterns, while deeper MLPs with differentiable activations successfully capture the non-linear structure.

Activation functions strongly influence convergence speed and learning effectiveness. Sigmoid activations suffer from vanishing gradients, resulting in slower convergence. ReLU activations enable faster learning due to stronger gradient flow for active neurons.

3 Conclusion

I understood the impact of network depth and activation functions on learning non-linear decision boundaries using the make_circles dataset.

The basic perceptron and step-activation MLPs failed to model non-linear patterns due to linear decision boundaries and lack of differentiability. Introducing hidden layers with differentiable activation functions enabled effective learning of complex structures.

Overall, the experiments highlight that both network depth and appropriate

activation functions are essential for efficient learning and stable convergence in neural networks.